

Manuel Utilisateur

Équipe 51 - groupe 9

19 janvier 2026

1 Introduction

Ce document constitue le manuel utilisateur du compilateur Deca développé par notre équipe dans le cadre du projet génie logiciel. Ce compilateur a pour objectif de traduire des programmes écrits en langage Deca vers du code assembleur IMA. Ce manuel se concentre sur les particularités et les limitations de notre implémentation, et il n'a pas pour vocation de présenter ou remplacer les spécifications détaillées du langage. On y trouve notamment :

- Les limitations connues du compilateur.
- Une liste exhaustive des messages d'erreurs que le compilateur peut retourner.
- Une présentation brève de l'extension.

2 Précisions sur l'implémentation du compilateur

2.1 Parties du langage mal implémentées

Certaines fonctionnalités du langage n'ont pas été entièrement implémentées ou présentent des comportements incorrects :

- La méthode **Object.equals** n'est pas fonctionnelle.
- Les instructions **print** appliquées aux types `int` et `float` présentent des dysfonctionnements.
- La gestion de plusieurs paramètres dans les instructions **printx** et **printlnx** est incorrecte.
- Des problèmes de **cast** depuis des instances de type `Object` apparaissent lors de la génération de code.
- L'utilisation de **this** dans les méthodes n'est pas correctement prise en charge.
- Erreurs d'exécution spécifiques (IMA)

STORE : Pas une adresse memoire

Cette erreur critique survient généralement lors de l'initialisation d'un objet.

- **Contexte** : Tentative d'écriture d'un champ ou d'une méthode dans le tas.
- **Diagnostic** : Le code assembleur généré a alloué une zone mémoire trop courte pour l'objet (ex : `NEW #1` au lieu de `NEW #2` ou plus). L'instruction **STORE** tente d'écrire à un index (ex : `1(R2)`) qui n'a pas été réservé.
- **Solution** : Il s'agit d'une limitation actuelle du compilateur sur le calcul de la taille des objets. Tentez de simplifier la structure de la classe ou de réduire le nombre de champs pour contourner le problème.

En revanche, les phases de compilation amont sont pleinement opérationnelles : Le lexeur, le parseur et l'analyse contextuelle sont entièrement fonctionnels, d'après nos tests.

Concernant les options du compilateur :

Les options **-b**, **-r**, **-d**, **-p** et **-v** fonctionnent correctement.

Les options **-n** et **-P** sont fonctionnelles, mais leur comportement est partiellement entravé par les limitations de la phase de génération de code.

2.2 Impact sur l'utilisateur

Les limitations identifiées affectent principalement la phase de génération de code, ce qui peut entraîner :

- Des résultats incorrects ou incomplets à l'exécution, notamment lors de l'utilisation des instructions d'affichage (`print`, `printx`, `printlnx`, `print`, `printx` et `printlnx`), elles sont opérationnelles mais gèrent mal certains mélanges de types.
- Une impossibilité d'utiliser correctement certaines fonctionnalités orientées objet, telles que `Object.equals`, les casts impliquant `Object`, ou l'accès à l'instance courante via `this`.
- Une utilisation restreinte de certaines options du compilateur (`-n`, `-P`) lorsque le programme compilé fait appel à des constructions affectées par la génération de code.

Cependant, l'utilisateur peut toujours :

- Écrire et analyser correctement des programmes grâce à un front-end de compilation robuste (lexeur, parseur, analyse contextuelle).
- Exploiter pleinement les options de diagnostic, de configuration et de verbosité du compilateur, indépendamment des limitations identifiées dans la partie génération de code.

3 Guide pour les erreurs

3.1 Liste des messages d'erreur possibles

- Nombre d'arguments incorrect
- `instanceof` ne s'applique qu'à un objet, `instanceof` attend un type classe
- Type `void` interdit pour les variables
- Variable `[nom_variable]` déjà déclarée
- Nouvelle variable en doublon : Type `[nom_type]` inconnu pour le paramètre
- Un paramètre ne peut pas être de type `void`
- La signature de la méthode redéfinie doit avoir le même nombre de paramètres
- Les types des paramètres doivent correspondre à ceux de la méthode héritée
- Le type de retour de la méthode redéfinie doit être un sous-type du type de la super-classe
- Méthode `[nom_methode]` déjà définie dans cette classe
- Un champ ne peut pas être de type `void`
- `[nom_champ]` existe déjà dans la super-classe mais n'est pas un champ
- La super-classe `[nom_super_classe]` n'existe pas
- `[nom_super_classe]` n'est pas une classe

- Cast incompatible entre [type_classe] et [type_expression]
- Type non imprimable dans une instruction `print` : [type]
- Les opérandes arithmétiques doivent être `int` ou `float`, pas [type_gauche] et [type_droit]
- Affectation entre des types invalides : attendu [type_attendu], trouvé [type_trouve]
- La condition d'un `if` ou `else` doit être de type booléen : [type_condition]
- Type de retour de la méthode incorrect : attendu [type_retour_attendu], trouvé [type_retour_trouve]
- Appel de méthode sur un type non-classe
- Méthode [nom_methode] inexistante
- [nom_methode] n'est pas une méthode
- Le modulo nécessite deux entiers, pas [type_gauche] et [type_droit]
- La classe [nom_classe] est inconnue
- [nom_classe] n'est pas une classe
- Opération `not` [operande] invalide
- `return` avec un `void` interdit
- La sélection s'applique uniquement aux classes
- Champ [nom_champ] inexistant dans [type_classe]
- Accès impossible au champ protégé [nom_champ]
- Accès protégé invalide
- Le type de l'expression n'est pas un sous-type de la classe courante
- `this` interdit dans le programme principal
- Opération - [operande] invalide
- La condition d'un `while` doit être booléenne
- [nom_fichier] : fichier `include` introuvable
- Le champ n'a pas de nom
- `Float` n'est pas un nombre valide
- Inclusion circulaire pour le fichier [nom_fichier]
- Erreur de reconnaissance de token à : [char]
- Aucune alternative viable à l'entrée [char]

4 Extension

Nous avons enrichi la bibliothèque `Math` avec des fonctions trigonométriques , à savoir : `cos` , `sin` , `acos` , `asin` , `atan` et `ulp`.

Cette extension permet d'effectuer des calculs scientifiques complexes de Deca .

Et pour ce faire , on a implémenté plusieurs algorithmes et méthodes ,comme Cordic , Réduction d'intervalle , Table de constantes , utilisation des codes assembleur à l'intérieur des classes ..

4.1 Mode opératoire

- **ULP** : La fonction `ulp` (unit in the last place) représente la valeur du bit de poids faible de la mantisse , on l'a implémenté en utilisant une méthode de normalisation itérative, permettant de s'affranchir des manipulations de bas niveau tout en garantissant une précision à l'unité près sur l'exposant tout en évitant l'utilisation des masques sur les bits ,qui est impossible en deca.

pour commencer , on traite les cas particuliers comme zero et NaN , et par la suite :

On détermine l'exposant par normalisation : tant que la val > 2 , on divise par 2 successivement jusqu'à ce que le nombre soit compris dans $[1,2[$, et donc l'exposant vaut le nombre de divisions qu'on a fait (cet exposant est positif dans ce cas)

* et si val < 2 , on multiplie par 2 jusqu'à atteindre $[1,2[$, de même l'exposant négatif est le nombre de multiplication qu'on vient de faire.

Calcul de décalage de précision : En simple précision (32 bits), la mantisse possède 23 bits.

c'est-à-dire La valeur de l'ULP est 2^{exp-32} . notre algorithme vérifie si on descend en dessous du seuil des nombres normalisés ($exp < -103$)pour retourner la valeur minimale possible

Reconstruction de la puissance de 2 : notre algorithme reconstruit la valeur flottante voulu par une boucle de multiplications ou de divisions successives par 2.0, ce qui garantit que le résultat final est une puissance de 2 exacte sans erreur d'arrondi intermédiaire.

- **Sin (et cos) :** ces deux fonctions sont les plus difficile à implémenter et à gérer , surtout comment reduire les angles , par exemple si on prend en entre une gros valeur comme 100000 , pour rendre cette valeur dans $[0,2\pi]$, on perd beaucoup de précision à cause des divisions qu'on fait , mais avec des optimisations qu'on a fait surtout l'ajout des valeurs pré-calculé , on on stock une liste des multiples de 2π , et avec une **recherche binaire** , on atteint de facon efficace la valeur voulu avec plus de précision

Après réduction de x en $[0,\pi/2]$;

Choix de la méthode :

- si $x_red < TRES_PETIT = 0.001$: result = x_red ;
- si $x_red < PETIT = 0.1$: result = $tableSin(x_red)$;on appelle la methode tableSin et tableCos (une table pré-calculé avec une grille de valeurs entre 0 et $\pi/2$, liées a leurs Cos et sin , ces valeurs sont prises par le biais de java.lang.Math.cos et java.lang.Math.sin suivi d'une interpolation lineaire (ou avec une interpolation polynomiale comme Hermite , "on est entrain de comparer ces deux methodes" .
- si $x_red > 1,471$: donc res = $\pi/2 - x_red$; dans ce cas , on fait une conversion ,comme $\sin(\pi/2 - x) = \cos(x)$, donc en fonction de x , on determine la meilleur option
 - si res < 0.001 : result = 1.0f ; dans ce cas $\cos(res)$ 1
 - si res $< PETIT$: result = $tableCos(res)$; dans ce cas on appel la tableCos (meme principe que tableSin)
- sinon , on utilise l'algorithme **CORDIC** result = $CordicSin(x_red)$; , l'idée ce ce algorithme est d'approcher les valeurs cos et sin ,par des suites qui convergent , et ces suites depend des valeurs de $atan(2^{-i})$, pour i de 0 à 24 (un bon compromis entre efficacite et respect d'environnement car il faut par faire trop de calculs inutile ; ainsi qu'en (float 32 bits) , la mantisse est en 23 bits , donc c'est pas possible de dépasser les 24 iterations ; on stock une tables des valeurs pour $atan(2^{-i})$ et aussi les $pow2i = 2^{-i}$

- pour la fonction $\cos$, on traite les cas particuliers de zero , et Nan et en suite on appelle le $\sin$ à $(\pi/2 - x)$
- **Atan** : pour la fonction Atan , on utilise une combinaison entre des **series de Taylor** et l'algorithme **CORDIC en mode vectorielle** avec le meme principe qu'en $\sin$ et $\cos$, on conserve des tables de $\text{atan}(2^{-i})$ et 2^{-i}
- **ASin (et Acos)** : la fonction $\text{Acos}(x) = \pi/2 - \text{Asin}(x)$
 donc ce qu'on fait c'est juste d'implementer la fonction Asin et pour Acos ,on appelle directement $\text{Asin}(x)$. pour la fonction Asin , on utilise la fonction Atan , car on sait que $\text{atan}(x/\sqrt{1-x^2})$, mais avant on traite tous les cas particuliers , en plus la gestion de x proche de 1 , là ou le denominateur s'annule , aussi on utilise l'algorithme de Newton (méthode de Newton-Raphson) avec deux versions (l'une pour calculer ou approcher la racine de $x > 0$, et l'autre version pour calculer l'inverse de la racine (on est entrain de tester la meilleur option , d'apres les tests qu'on a fait on prefere calculer la racines et ensuite l'inverser ,plutot que de calculer directement l'inverse de la racine)

4.2 Limitations

- Le compilateur deca ne permet pas tous les méthodes et fonctionnalités nécessaires comme la gestion explicite des bits , et déclaration des variables dans les `while` , et dans les expressions
- Difficulté dans la gestion de la pile , donc plus de risque d'erreurs
- float 32 bits , présente plus de contraintes , et rendre la mission plus difficile , surtout qu'on veut réduire des angles (division successive
- Gestion de Cache pour les tables de Cos sin et Cordic
-